

VUE.JS

Aula 7: Pinia e Projeto Final

Estado compartilhado entre componentes, e um projeto final juntando tudo que vimos

O que vamos ver hoje

1

Por que Pinia?

O problema de compartilhar dados entre componentes distantes

2

Instalando e criando uma store

npm install pinia e a primeira store

3

state, getters e actions

As três partes de uma store

4

Usando a store nos componentes

Lendo e alterando o estado compartilhado

5

Projeto final

Juntando Vue Router, API e Pinia em uma aplicação completa

AULA 7 / 7

Última aula! Hoje fechamos o módulo com a última peça que faltava e um projeto que junta tudo que você aprendeu.

Revisão rápida da Aula 6

Antes de seguir, lembre o que já sabemos fazer:

`fetch()`

Faz uma requisição HTTP e devolve uma Promise

`async/await`

Escreve código assíncrono no estilo síncrono

`try/catch`

Trata erros sem quebrar a aplicação

`carregando/erro`

Estados que guiam o que a tela mostra

Hoje: nossos dados vivem dentro de um componente e somem quando ele desmonta. Vamos aprender a guardar estado num lugar compartilhado.

Por que Pinia?

props e emits resolvem pai/filho direto. Mas e quando dois componentes distantes precisam do mesmo dado?

Sem estado compartilhado

- ✗ Para levar um dado do avô até o neto, cada componente no meio precisa repassar a prop
- ✗ Isso é chamado de "prop drilling" — e piora conforme a árvore cresce
- ✗ Componentes irmãos não têm como se comunicar diretamente
- ✗ Duplicar a mesma lista de tarefas em telas diferentes gera dados desincronizados

Com Pinia

- ✓ O estado compartilhado mora em um lugar só: a store
- ✓ Qualquer componente pode ler ou alterar a store diretamente, sem repassar props
- ✓ Componentes irmãos e telas diferentes sempre veem o mesmo dado atualizado
- ✓ Continua sendo reativo: mudou na store, a tela atualiza sozinha

Pinia é a biblioteca oficial de gerenciamento de estado do Vue — o sucessor do Vuex.

Instalando e criando uma store

Assim como o router, o Pinia se conecta à aplicação uma vez em main.js.

```
$ npm install pinia
```

```
// src/main.js
import { createPinia } from "pinia";

const app = createApp(App);
app.use(router);
app.use(createPinia());
app.mount("#app");
```

```
// src/stores/tarefas.js
import { defineStore } from "pinia";

export const useTarefasStore = defineStore(
  "tarefas",
  {
    state: () => ({
      lista: []
    })
  }
);
```

defineStore("tarefas", ...) cria uma store com o nome "tarefas" — esse nome identifica a store internamente no Pinia.

state, getters e actions

Uma store Pinia tem três partes — e elas são bem parecidas com o que você já conhece de um componente.

Parte da store	Equivale a...	Para que serve
state	data()	Os dados guardados na store
getters	computed	Valores calculados a partir do state
actions	methods	Funções que alteram o state

Se você já entendeu data(), computed e methods, já entendeu 90% do Pinia — só muda onde essas peças moram.

Criando a store de tarefas, completa

Juntando state, getters e actions em uma store pronta para uso.

```
// src/stores/tarefas.js
import { defineStore } from "pinia";
export const useTarefasStore = defineStore("tarefas", {
  state: () => ({
    lista: []
  }),
  getters: {
    restantes: (state) => state.lista.filter((t) => !t.feita).length
  },
  actions: {
    adicionar(texto) {
      this.lista.push({ id: Date.now(), texto, feita: false });
    },
    marcarFeita(id) {
      const tarefa = this.lista.find((t) => t.id === id);
      tarefa.feita = true;
    }
  }
});
```

Usando a store em um componente

Qualquer componente pode importar a store, guardar em `data()` e usá-la no template.

```
<!-- Tarefas.vue -->
<template>
  <p>{{ store.restantes }} restantes</p>
  <ul>
    <li v-for="t in store.lista" :key="t.id">
      {{ t.texto }}
      <button @click="store.marcarFeita(t.id)">
        Concluir</button>
    </li>
  </ul>
</template>
<script>
import { useTarefasStore } from "../stores/tarefas";
export default {
  data() {
    return { store: useTarefasStore() };
  }
};
</script>
```

Repare

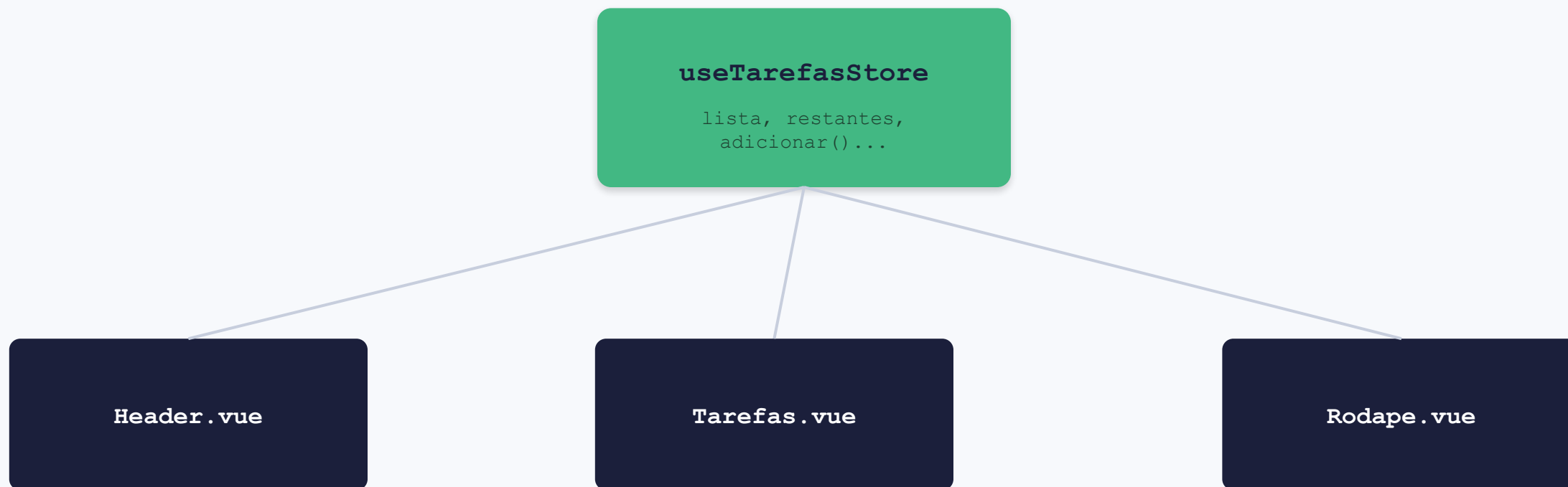
`useTarefasStore()` devolve a store — a mesma instância em qualquer componente que chamar essa função

`store.lista` e **`store.restantes`** funcionam como qualquer dado reativo no template

Chamar uma action (**`store.marcarFeita(id)`**) é como chamar um method comum

Um estado, vários componentes

Diferente de props/emits, a store não segue a árvore de componentes — qualquer um pode acessá-la direto.



Três componentes que não têm relação de pai/filho entre si, todos lendo e alterando a mesma store — sem passar uma única prop.

Visão geral do projeto final

Todas as peças das sete aulas encaixadas em uma única aplicação.

- | | | |
|---|------------------------------|--|
| 1 | App.vue + Router | Navegação entre Home, Tarefas e Sobre (Aula 5) |
| 2 | Store de tarefas | state, getters e actions compartilhados (hoje) |
| 3 | Busca inicial na API | created() assíncrono preenche a store (Aula 6) |
| 4 | Tarefaltem.vue | Componente filho com props e emits (Aula 3) |
| 5 | Ciclo de vida e watch | Logs e reações a mudanças de estado (Aula 4) |

Lista de tarefas completa

O que construir

- Crie uma store useTarefasStore com state (lista), getter (restantes) e actions (adicionar, marcarFeita)
- Na tela Tarefas.vue, busque as tarefas iniciais de uma API no created()
- Salve o resultado da API na store, não em data() local
- Renderize cada tarefa com um componente filho TarefaItem.vue (props + emits)
- O clique em "Concluir" emite um evento que chama a action da store
- Use o router para navegar entre Home, Tarefas e Sobre

```
<!-- TarefaItem.vue -->
<template>
  <li>
    {{ texto }}
    <button @click="$emit('concluir')">
      Concluir</button>
  </li>
</template>

<script>
export default {
  props: ["texto"],
  emits: ["concluir"]
};
</script>
```

Recapitulando o curso

As sete aulas, do zero até uma aplicação completa.

- | | | |
|---|-----------------------------|--|
| 1 | Primeiros passos | CDN, createApp, data() e interpolação |
| 2 | Reatividade e listas | v-model, v-for, methods e computed |
| 3 | Componentização | props e emits entre componentes |
| 4 | Ciclo de vida e Vite | created, mounted, watch e Single File Components |
| 5 | Vue Router | Navegação entre páginas |
| 6 | Consumindo APIs | fetch, Promises e async/await |
| 7 | Pinia | Estado compartilhado e projeto final |

VUE.JS

Parabéns pelo módulo concluído!

Você saiu de um <script> via CDN até uma aplicação com rotas, API e estado compartilhado.

Para continuar evoluindo:

- Composition API (script setup) — outra forma de organizar a lógica dos componentes
- TypeScript — tipagem para projetos maiores
- Nuxt — framework Vue com renderização no servidor
- Testes automatizados com Vitest